FOSSEE Summer Fellowship Screening Task Report

On

OSDAG: Parametric 3D CAD Model of a Steel Girder Bridge

Submitted by:

Shwaas Adlakha

3rd Year B.Tech. C.S.E. Student
School of Computing Science & Engineering
VIT Bhopal University, Bhopal, Madhya Pradesh

April 13, 2026

OSDAG Software:

Osdag (Open steel design and graphics) is a cross-platform, free/libre and open-source software designed for the detailing and design of steel structures based on the Indian Standard IS 800:2007. It allows users to design steel connections, members, and systems through an interactive graphical user interface (GUI) and provides 3D visualizations of designed components. The software enables easy export of CAD models to drafting tools for construction/fabrication drawings, with optimized designs following industry best practices [1, 2, 3]. Built on Python and several Python-based FLOSS tools (e.g., PyQt and PythonOCC), Osdag is licensed under the GNU Lesser General Public License (LGPL) Version 3.

About this report:

This report documents the work done for the Osdag FOSSEE Fellowship screening task, which required building a parametric 3D CAD model of a short-span steel girder bridge using the pythonOCC library. The task evaluated the ability to write modular, reusable code; apply engineering geometry logic; and model reinforced concrete components with structural detailing awareness.

Screening Task

Problem Statement:
Create a parametric, modular 3D CAD model of a short-span steel girder bridge using the pythonOCC (*pythonocc-core*) library. The model should be assembled from component factories (girders, deck slab, piers, pier cap, piles, pile cap, reinforcement bars) so each part is adjustable by changing parameters. The final deliverable is a runnable *bridge_model.py* script that builds, visualizes, and optionally exports the bridge geometry. Concrete parts should be semi-transparent so internal steel reinforcement bars are visible.

Objectives:
• Implement a reusable, parameter-driven script (*bridge_model.py*) that assembles a complete short-span bridge model from smaller component factories.
• Deliver visual output (OCC display), tests, documentation, and packaged submission files.

Summary:
The objective was to create a fully parametric, modular 3D CAD model of a short-span steel girder bridge using the pythonOCC (*pythonocc-core*) library. The model had to be assembled from component factories (girders, deck slab, piers, pier cap, piles, pile cap, and reinforcement bars) such that every dimension is adjustable through named parameters.
The deliverable was a runnable *bridge_model.py* script that builds, visualizes, and optionally exports the bridge geometry, with concrete parts rendered semi-transparent so that internal steel reinforcement bars are visible.

The bridge components required were:
- Main longitudinal steel girders (I-sections), three or more
- Concrete deck slab and parapets
- Concrete piers (circular columns) and trapezoidal pier caps
- Pile cap (rectangular) and piles (circular)
- Steel reinforcement bars inside the deck slab¶

Contents

1. **Software Architecture & Modular Design**
2. **Coordinate System & Spatial Mathematic**
3. **Parametric Control and CLI**
4. **Component Breakdown**
5. **Visualization & Output**
6. **Conclusions**

1. Software Architecture & Modular Design

The script is structured around a three-layer factory pattern that cleanly separates geometry creation, assembly logic, and final orchestration.

Layer 1: Primitive Factories return raw, unpositioned *TopoDS_Shape* solids. Each factory takes only geometric parameters as input and has no knowledge of where the shape will be placed in the global model.

Layer 2: Assembly Functions call the factories, apply translation transforms to position each component, and group them into *TopoDS_Compound* objects. A compound is OCCT's container for multiple shapes treated as a single entity.

Layer 3: Main Orchestrator ***__main__*** calls the assembly functions, computes inter-component Z-positions, passes all shapes to the display renderer, and applies color and transparency settings.

A key architectural decision is that the same ***create_circular_pier()*** *factory serves three structurally distinct roles: the main pier column, the individual foundation piles, and the cylindrical rebar bars inside the deck slab. This is the factory pattern's central benefit, geometry type is decoupled from structural role, eliminating redundant code.*

Wherever multiple shapes belong to the same sub-system, they are collected into a *TopoDS_Compound* using the *BRep_Builder* pattern:

```
compound_shape = TopoDS_Compound()
builder = BRep_Builder()
builder.MakeCompound(compound_shape)
builder.Add(compound_shape, positioned_shape)
return compound_shape
```

** This pattern is used in build_foundation(), build_substructure(), build_girders(), build_parapets(), and create_rebar_grid_for_deck().*

2. Coordinate System & Spatial Mathematics

Axis convention: X runs longitudinally along the span, Y runs transversely across the deck width, and Z runs vertically upward. The global origin (0, 0, 0) is set at the longitudinal midspan centre at the base of the pier column (ground level/top of pile cap)

The vertical Z-stacking chain defines each component's position from the one below it:

```
Z = 0
Z = pier_height
Z = pier_height + pier_cap_depth
Z = pier_height + pier_cap_depth + girder_d
Z = pier_height + pier_cap_depth + girder_d + deck_thickness
```

Translation transforms are handled by the *move_shape()* utility, which wraps OCCT's transform pipeline. Internally this is a 4×4 homogeneous matrix applied to every vertex of the *TopoDS_Shape*:

```
def move_shape(shape, move_x, move_y, move_z):
    translation_vector = gp_Vec(move_x, move_y, move_z)
    transformation = gp_Trsf()
    transformation.SetTranslation(translation_vector)
    transformer = BRepBuilderAPI_Transform(shape, transformation,
True)
    return transformer.Shape()
```

* The ***True*** flag in *BRepBuilderAPI_Transform* copies the shape before transforming it. This is essential, it allows the single master substructure compound to be placed at both the left and right support positions without memory aliasing.

Rotation transforms are used for the rebar grid. Cylinders from *BRepPrimAPI_MakeCylinder* default to Z-axis orientation. Rebar bars must lie along X or Y, so axis-angle rotations are applied using *gp_Ax1*:

```
# Rotate cylinder to lie along Y (transverse bars)
x_axis = gp_Ax1(gp_Pnt(0,0,0), gp_Dir(1,0,0))
rotation.SetRotation(x_axis, -math.pi / 2.0)

# Rotate cylinder to lie along X (longitudinal bars)
y_axis = gp_Ax1(gp_Pnt(0,0,0), gp_Dir(0,1,0))
rotation.SetRotation(y_axis, math.pi / 2.0)
```

3. Parametric Control & CLI

Every geometric dimension is exposed as a named CLI flag using Python's *argparse* module, with engineering-appropriate defaults.
Running python *bridge_model.py --help* prints the full parameter list. Changing a single argument rebuilds the entire model without touching any geometry logic.

```
parser = argparse.ArgumentParser(
    description="Parametric 3D CAD Model of a Steel Girder Bridge"
)
parser.add_argument("--span_length", type=float, default=12000,
                    help="Total clear span length (mm)")
parser.add_argument("--n_girders",   type=int,   default=3,
                    help="Number of main longitudinal girders")
```

Example Usage:

```
# Default 12 m span bridge
python bridge_model.py

# Custom span with 4 girders and taller pier
python bridge_model.py --span_length 18000 --n_girders 4 --pier_height 5000
```

* *One derived parameter is girder spacing, if left at 0, it is auto-computed as*
girder_spacing = deck_width / n_girders
preventing contradictory inputs.

Full Parameter Reference

span_length	12000 mm	Total clear span length
deck_width	7000 mm	Total deck width (transverse)
deck_thickness	250 mm	Deck slab thickness
pier_diameter	800 mm	Circular pier column diameter
pier_height	3000 mm	Pier column height (ground to cap)
pier_cap_length	1000 mm	Pier cap thickness along span
pier_cap_top_width	6900 mm	Pier cap top edge width
pier_cap_bottom_width	6000 mm	Pier cap bottom edge width
pier_cap_depth	600 mm	Pier cap vertical depth
girder_spacing	deck_width / n_girders	Girder c/c spacing
girder_d	900 mm	Girder section depth (d)
girder_bf	300 mm	Girder flange width (bf)
girder_tf	16 mm	Girder flange thickness (tf)
girder_tw	200 mm	Girder web thickness (tw)
pile_cap_length	2200 mm	Pile cap length
pile_cap_width	2200 mm	Pile cap width
pile_cap_depth	600 mm	Pile cap depth
pile_diameter	400 mm	Individual pile diameter
pile_length	5000 mm	Individual pile depth
pile_spacing	1300 mm	Pile centre-to-centre spacing
rebar_main_diameter	20 mm	Longitudinal rebar diameter
rebar_spacing_longitudinal	150 mm	Longitudinal rebar spacing
rebar_transverse_diameter	20 mm	Transverse rebar diameter
rebar_spacing_transverse	200 mm	Transverse rebar spacing
rebar_cover	40 mm	Concrete clear cover to rebar

```
(osdag) adlakhashwaas@Shwaass-MacBook-Air ~ % python3 /Users/adlakhashwaas/Documents/Life/Software/Projects/FOSSEE\ Summer\ Fellowship\ 2026/Osdag/bridge-model/bridge_model.py -h
usage: bridge_model.py [-h] [--span_length SPAN_LENGTH] [--deck_width DECK_WIDTH] [--deck_thickness DECK_THICKNESS] [--pier_diameter PIER_DIAMETER] [--pier_height PIER_HEIGHT]
                       [--pier_cap_length PIER_CAP_LENGTH] [--pier_cap_top_width PIER_CAP_TOP_WIDTH] [--pier_cap_bottom_width PIER_CAP_BOTTOM_WIDTH] [--pier_cap_depth PIER_CAP_DEPTH]
                       [--n_girders N_GIRDERS] [--girder_spacing GIRDER_SPACING] [--girder_d GIRDER_D] [--girder_bf GIRDER_BF] [--girder_tf GIRDER_TF] [--girder_tw GIRDER_TW]
                       [--pile_cap_length PILE_CAP_LENGTH] [--pile_cap_width PILE_CAP_WIDTH] [--pile_cap_depth PILE_CAP_DEPTH] [--pile_diameter PILE_DIAMETER] [--pile_length PILE_LENGTH]
                       [--pile_spacing PILE_SPACING] [--rebar_main_diameter REBAR_MAIN_DIAMETER] [--rebar_spacing_longitudinal REBAR_SPACING_LONGITUDINAL]
                       [--rebar_transverse_diameter REBAR_TRANSVERSE_DIAMETER] [--rebar_spacing_transverse REBAR_SPACING_TRANSVERSE] [--rebar_cover REBAR_COVER]
```

```
(osdag) adlakhashwaas@Shwaass-MacBook-Air ~ % python3 /Users/adlakhashwaas/Documents/Life/Software/Projects/FOSSEE\ Summer\ Fellowship\ 2026/Osdag/bridge-model/bridge_model.py --help
usage: bridge_model.py [-h] [--span_length SPAN_LENGTH] [--deck_width DECK_WIDTH] [--deck_thickness DECK_THICKNESS] [--pier_diameter PIER_DIAMETER] [--pier_height PIER_HEIGHT]
                       [--pier_cap_length PIER_CAP_LENGTH] [--pier_cap_top_width PIER_CAP_TOP_WIDTH] [--pier_cap_bottom_width PIER_CAP_BOTTOM_WIDTH] [--pier_cap_depth PIER_CAP_DEPTH]
                       [--n_girders N_GIRDERS] [--girder_spacing GIRDER_SPACING] [--girder_d GIRDER_D] [--girder_bf GIRDER_BF] [--girder_tf GIRDER_TF] [--girder_tw GIRDER_TW]
                       [--pile_cap_length PILE_CAP_LENGTH] [--pile_cap_width PILE_CAP_WIDTH] [--pile_cap_depth PILE_CAP_DEPTH] [--pile_diameter PILE_DIAMETER] [--pile_length PILE_LENGTH]
                       [--pile_spacing PILE_SPACING] [--rebar_main_diameter REBAR_MAIN_DIAMETER] [--rebar_spacing_longitudinal REBAR_SPACING_LONGITUDINAL]
                       [--rebar_transverse_diameter REBAR_TRANSVERSE_DIAMETER] [--rebar_spacing_transverse REBAR_SPACING_TRANSVERSE] [--rebar_cover REBAR_COVER]

Parametric 3D CAD Model of a Steel Girder Bridge

options:
  -h, --help            show this help message and exit
  --span_length SPAN_LENGTH
                        Total clear span length (mm)
  --deck_width DECK_WIDTH
                        Total deck width (mm)
  --deck_thickness DECK_THICKNESS
                        Thickness of the deck slab (mm)
  --pier_diameter PIER_DIAMETER
                        Diameter of the circular pier (mm)
  --pier_height PIER_HEIGHT
                        Height of the pier from pile cap to pier cap (mm)
  --pier_cap_length PIER_CAP_LENGTH
                        Length (thickness) of the pier cap (mm)
  --pier_cap_top_width PIER_CAP_TOP_WIDTH
                        Top width of the pier cap (mm)
  --pier_cap_bottom_width PIER_CAP_BOTTOM_WIDTH
                        Bottom width of the pier cap (mm)
  --pier_cap_depth PIER_CAP_DEPTH
                        Depth (height) of the pier cap (mm)
  --n_girders N_GIRDERS
                        Number of main longitudinal girders
  --girder_spacing GIRDER_SPACING
                        Center-to-center spacing of main girders (mm)
  --girder_d GIRDER_D   Depth of main girder I-section (mm)
  --girder_bf GIRDER_BF
                        Flange width of main girder (mm)
  --girder_tf GIRDER_TF
                        Flange thickness of main girder (mm)
  --girder_tw GIRDER_TW
                        Web thickness of main girder (mm)
  --pile_cap_length PILE_CAP_LENGTH
                        Length of the pile cap (mm)
  --pile_cap_width PILE_CAP_WIDTH
                        Width of the pile cap (mm)
  --pile_cap_depth PILE_CAP_DEPTH
                        Depth of the pile cap (mm)
  --pile_diameter PILE_DIAMETER
                        Diameter of individual piles (mm)
  --pile_length PILE_LENGTH
                        Length (depth) of individual piles (mm)
  --pile_spacing PILE_SPACING
                        Spacing between pile centers (mm)
  --rebar_main_diameter REBAR_MAIN_DIAMETER
                        Diameter of longitudinal rebar (mm)
  --rebar_spacing_longitudinal REBAR_SPACING_LONGITUDINAL
                        Spacing of longitudinal rebar (mm)
  --rebar_transverse_diameter REBAR_TRANSVERSE_DIAMETER
```

```
Last login: Mon Apr 13 17:58:33 on ttys000
adlakhashwaas@Shwaass-MacBook-Air ~ % conda activate osdag
(osdag) adlakhashwaas@Shwaass-MacBook-Air ~ % python3 /Users/adlakhashwaas/Documents/Life/Software/Projects/FOSSEE\ Summer\ Fellowship\ 2026/Osdag/bridge-model/bridge_model.py --n_girders 6
pyqt5 backend - Qt version 5.15.15
####### 3D rendering pipe initialisation #####
Display3d class initialization starting ...
Aspect_DisplayConnection created.
OpenGl_GraphicDriver created.
V3d_Viewer created.
AIS_InteractiveContext created.
V3d_View created
Graphic3d_Camera created
Graphic3d_StructureManager created
Cocoa window created.
Display3d class successfully initialized.
########################################
OpenGl information:
  GLvendor: Apple
  GLdevice: Apple M1
  GLversion: 2.1 Metal - 90.5
  GLSLversion: 1.20
  Max texture size: 16384
  Max FBO dump size: 16384x16384
  Max combined texture units: 16
  Max MSAA samples: 0
  Viewport: 2046x1524
  Window buffer: RGB8 ALPHA8 DEPTH32 STENCIL8
  ResolutionRatio: 1
  FBO buffer: GL_SRGB8_ALPHA8 GL_DEPTH24_STENCIL8
```

¶

4. Component Breakdown:

`create_rectangular_prism(width, height, length)`
Wraps BRepPrimAPI_MakeBox(length, width, height) to produce a box solid from the local origin. Used for the deck slab, pile cap, and parapets.

`create_circular_pier(diameter, height)`
Wraps BRepPrimAPI_MakeCylinder(radius, height) to produce a Z-axis cylinder. Used for pier columns, piles, and rebar bars.

`create_trapezoidal_pier_cap(length, top_width, bottom_width, depth)`
The most complex primitive. Constructs the hammerhead pier cap using OCCT's profile-extrusion workflow: four gp_Pnt points define the trapezoidal cross-section in the YZ plane, connected into edges with BRepBuilderAPI_MakeEdge, closed into a wire, filled into a planar face with BRepBuilderAPI_MakeFace, then swept along +X using BRepPrimAPI_MakePrism. This is the standard OCCT workflow for any custom prismatic solid and generalises to arbitrary cross-sections.

```
p1 = gp_Pnt(0, -bottom_width/2.0, 0)
p2 = gp_Pnt(0,  bottom_width/2.0, 0)
p3 = gp_Pnt(0,  top_width/2.0,   depth)
p4 = gp_Pnt(0, -top_width/2.0,   depth)
prism_maker = BRepPrimAPI_MakePrism(face, gp_Vec(length, 0, 0))
```

`create_rebar_grid_for_deck(...)`
Generates a complete orthogonal rebar grid.
Effective grid dimensions are reduced by 2 × cover from all slab edges. Transverse bars (along Y) are placed at the lower Z level; longitudinal bars (along X) sit on top, with $Z_offset = cover + transverse_diam + main_diam/2$, matching real structural detailing practice.

`build_substructure(...)`
Aggregates pile cap, piles, pier column, and pier cap into one compound. The left and right support substructures are both derived from a single master compound via two $move_shape()$ calls with ±X translations, applying a cap_inset correction to align pier cap faces with the span ends.

`build_girders(...)`
Positions n_girders I-section beams evenly across the deck width. The $start_y$ calculation centers the entire girder group on Y=0 regardless of the number of girders or spacing.

5. Visualization & Output

The OCC viewer is initialised via *init_ display*() from *OCC.Display.SimpleGui*, which launches a lightweight Qt-based 3D window. A white background is applied for clean presentation.

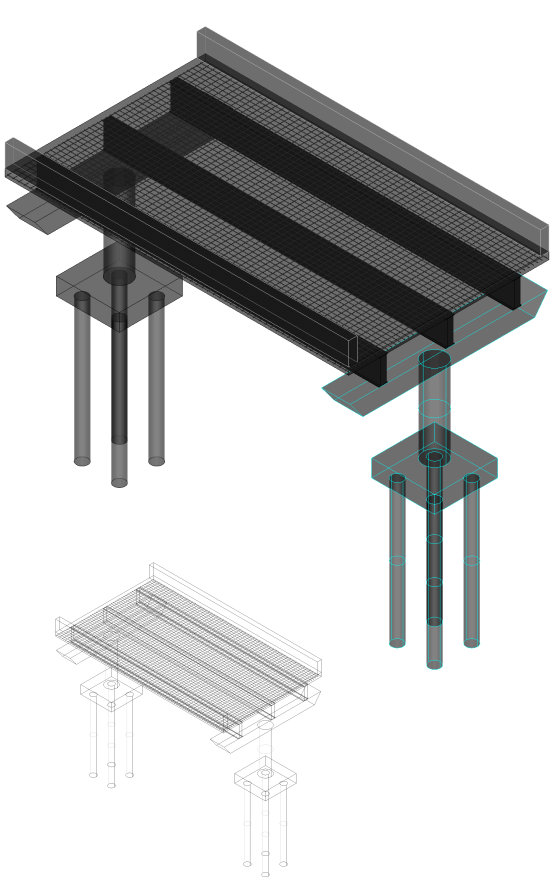
The visual system enforces a clear material distinction through transparency. Concrete components are displayed at transparency=0.4 (60% opaque), which is sufficient to show the slab geometry while revealing the rebar grid inside. Steel components are fully opaque. Both use OCCT's default grey shading under the viewer's lighting model.

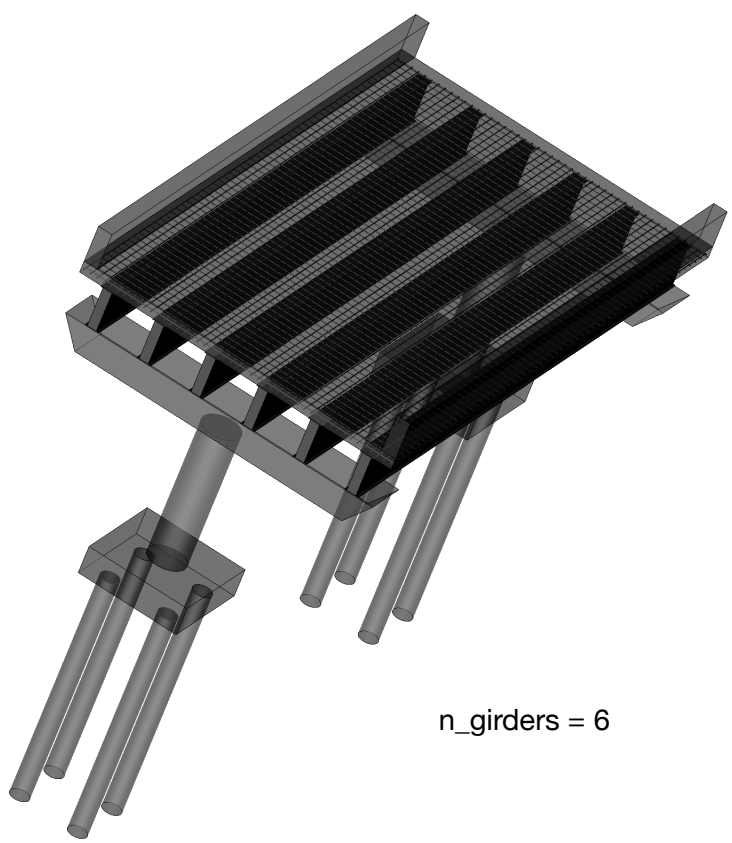
```
# Concrete: semi-transparent
display.DisplayShape(positioned_deck,   color="BLACK", transparency=0.4)
display.DisplayShape(substructure_left, color="BLACK", transparency=0.4)

# Steel: fully opaque
display.DisplayShape(all_girders,      color="BLACK")
display.DisplayShape(positioned_rebar, color="BLACK")
```

display.FitAll()

computes a bounding box over all displayed geometry and sets an isometric camera before *start_ display*() enters the GUI event loop.

¶





6. Conclusions:

6.1 Tasks Accomplished:

- Developed a complete parametric 3D CAD model of a short-span steel girder bridge using the pythonOCC library, assembled from modular component factories.
- Implemented a three-layer factory architecture separating geometry creation, assembly logic, and display orchestration.
- Exposed all bridge dimensions as CLI parameters using argparse, enabling full parametric control from the terminal without editing any source code.
- Modelled all required bridge components: I-section steel girders, reinforced concrete deck slab, circular piers, trapezoidal pier caps, rectangular pile cap, 2×2 pile foundation, parapets, and deck rebar grid.
- Applied correct structural detailing conventions including concrete cover, rebar layer stacking, and pier cap face alignment over the span.
- Rendered concrete elements semi-transparently to expose internal reinforcement in the 3D viewer.

6.2 Skills Developed:

- CAD automation with pythonOCC (OpenCASCADE): B-Rep solid modelling, profile-extrusion workflow, Boolean compounds, geometric transforms
- 3D spatial mathematics: homogeneous transformation matrices, axis-angle rotations, coordinate chaining in structural assemblies
- Parametric software design: CLI-driven architecture with argparse, factory design pattern, separation of concerns
- Engineering domain knowledge: structural hierarchy of a bridge (superstructure, substructure, foundation), rebar detailing conventions, I-section geometry